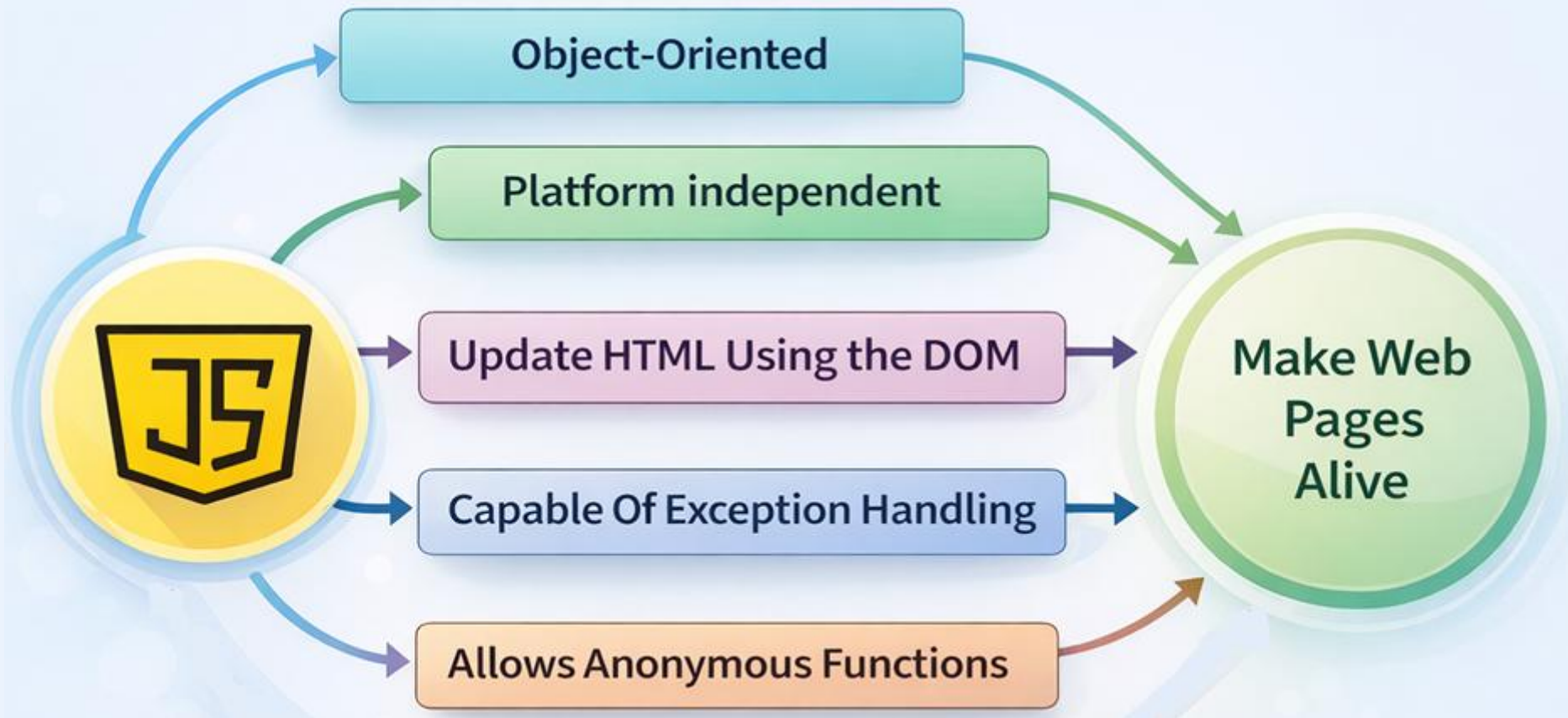


JavaScript

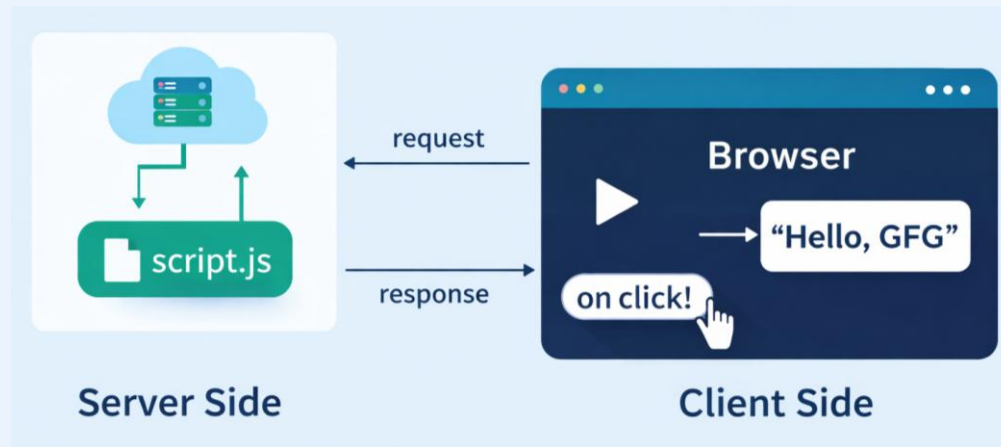
Introduction to JavaScript

- **JavaScript** is a versatile, dynamically typed programming language that brings life to web pages by making them interactive. It is widely used for building **interactive web applications** and integrates seamlessly with **HTML and CSS**.
- **Key characteristics of JavaScript:**
 - **Single-threaded language** – executes one task at a time.
 - **Interpreted language** – code is executed line by line without prior compilation.
 - **Dynamically typed** – the data type of a variable is determined at run-time.
 - Supports both **client-side and server-side development**
 - Provides a **rich ecosystem of libraries and frameworks** for modern web development.

Features of JavaScript



JavaScript execution



• Client-Side JavaScript :

- Executed within the **web browser** and mainly responsible for enhancing the **user interface and interaction**.
- Key roles include:
 - Manipulating webpage elements through the **Document Object Model (DOM)**
 - Handling user interactions such as **mouse clicks, keyboard input, and form submissions**
 - Updating webpage content dynamically without reloading the page
- Commonly used libraries and frameworks: **React.js, Angular**

• Server-Side JavaScript

- Runs on the **web server** and manages the **backend logic of web applications**.
- Major functionalities include:
 - Communicating with **databases**
 - Processing and storing application data
 - Handling **file operations and server responses**
- Popular technologies used: **Node.js runtime environment**, Frameworks such as **Express.js** for building scalable web applications

Programming Paradigms in JavaScript

- JavaScript supports multiple programming styles, allowing developers to solve problems in different ways.
- Two common paradigms are :

1. **Imperative programming** - In this approach, the programmer specifies **how a task should be performed**, controlling each step of the computation.

- Step-by-step instructions
- Explicit control flow using loops and conditions
- Common in procedural and object-oriented programming

```
let numbers = [1, 2, 3, 4];  
let doubled = [];  
  
for(let i = 0; i < numbers.length; i++) {  
    doubled.push(numbers[i] * 2);  
}  
console.log(doubled);
```

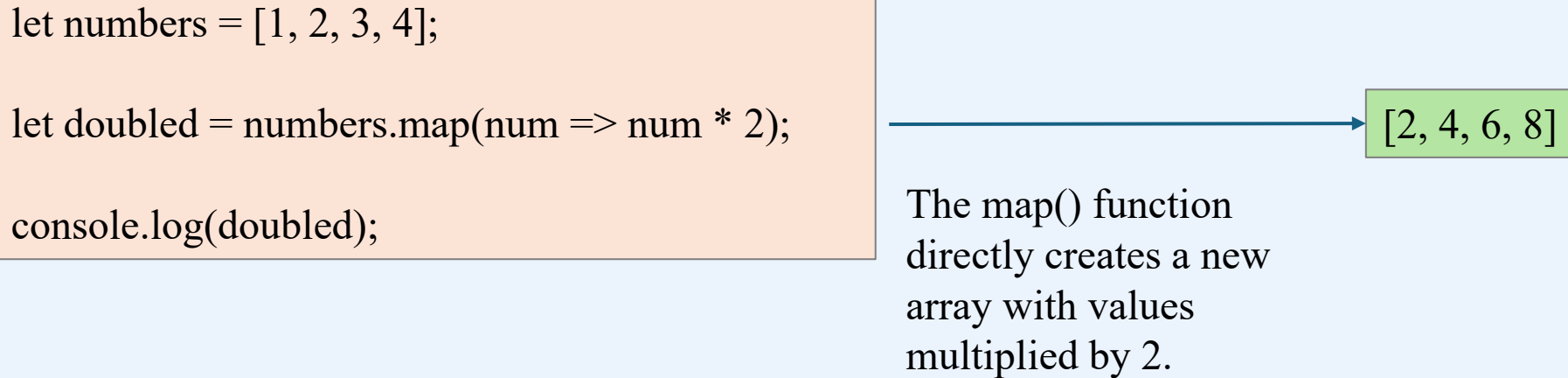
[2, 4, 6, 8]

Each element of the array is accessed using a loop and multiplied by 2.

Programming Paradigms in JavaScript

2. **Declarative programming** - In declarative programming, the focus is on **what result is required**, while JavaScript internally handles the steps.

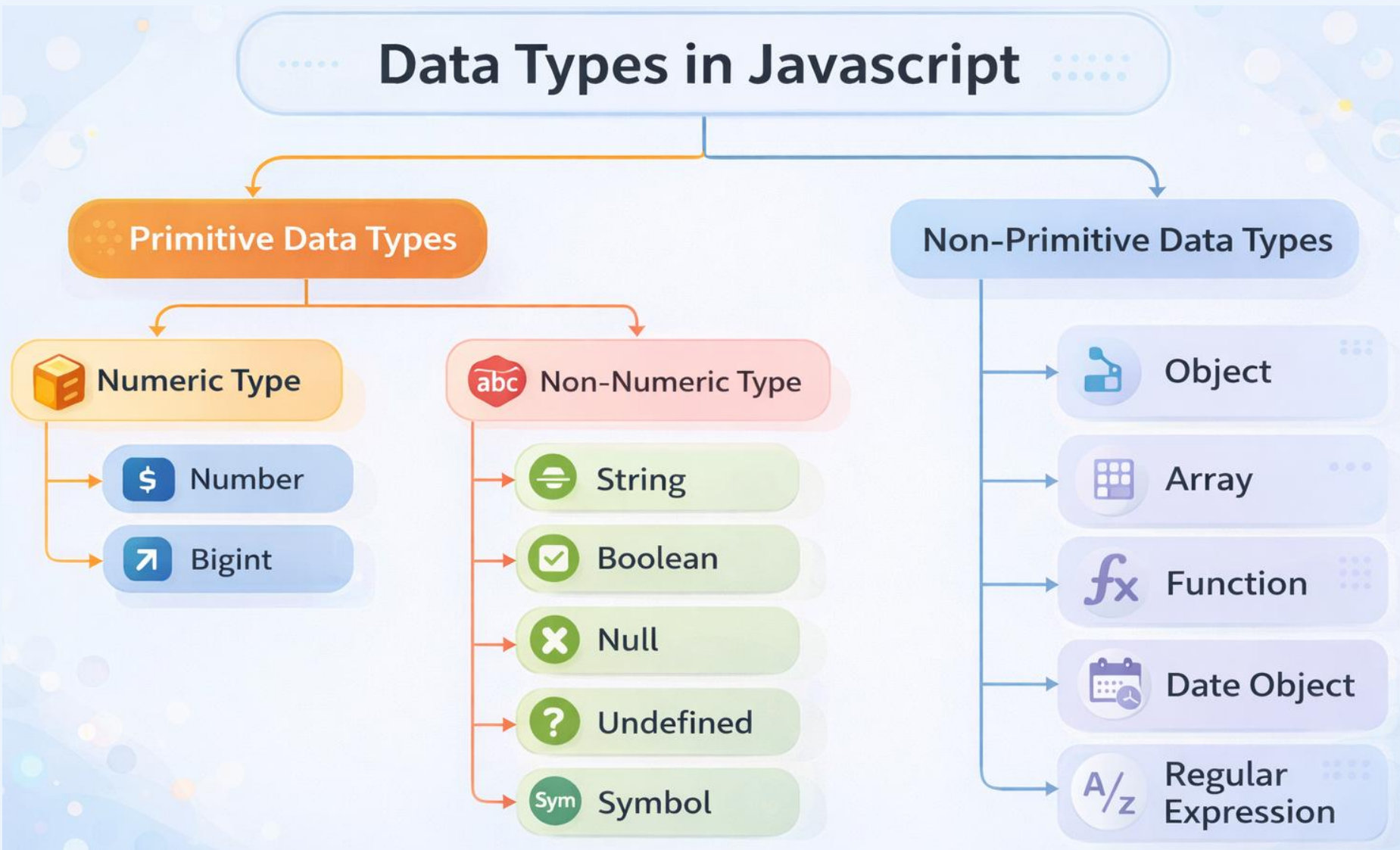
```
let numbers = [1, 2, 3, 4];  
  
let doubled = numbers.map(num => num * 2);  
  
console.log(doubled);
```



[2, 4, 6, 8]

The map() function directly creates a new array with values multiplied by 2.

Datatypes in JavaScript



Data Types in JavaScript

- **Primitive Data Types** - Primitive values are simple, immutable and stored directly in memory.

```
let num = 42;           // Number
let text = "Hello";     // String
let flag = true;        // Boolean
let empty = null;       // Null
let notAssigned;        // Undefined
let id = Symbol("id");  // Symbol
let big = 1234567890123456789n; // BigInt
```

- **Non-Primitive Data Types** - These types store **complex data structures**.

Object:

```
let person = {
  name: "Amit",
  age: 25
};
```

Array:

```
let colors = ["red", "green", "blue"];
```

Function:

```
function greet() {
  console.log("Welcome!");
}
```


JavaScript Operators

Arithmetic Operators

Expression	Output	Explanation
5 + 3	8	Adds two numbers
10 - 4	6	Subtracts second number from first
6 * 2	12	Multiplies two numbers
8 / 2	4	Divides one number by another
7 % 3	1	Returns remainder after division
2 ** 3	8	Exponentiation (2 raised to power 3)

Assignment Operators

Expression	Output	Explanation
let x = 10	10	Assigns value 10 to variable x
x += 5	15	Adds 5 to x and stores result in x
x -= 3	12	Subtracts 3 from x
x *= 2	24	Multiplies x by 2
x /= 4	6	Divides x by 4

Comparison Operators

Expression	Output	Explanation
5 > 3	true	Checks if 5 is greater than 3
4 < 2	false	Checks if 4 is smaller than 2
5 == "5"	true	Values are equal
5 === "5"	false	Values equal but data types different
6 != 3	true	Checks if values are not equal

String Operators

Expression	Output	Explanation
"Hello" + "World"	Hello World	Combines two strings
"Age: " + 25	Age: 25	Converts number to text and combines

Logical Operators : &&(AND), || (OR)and !(NOT)

Increment and Decrement Operators : x++, ++x, y--, --y

JavaScript Expression Evaluation

Expression	Output	Explanation
<code>console.log(null === undefined)</code>	false	null and undefined both represent empty values , but they are different data types . The strict equality operator (<code>===</code>) compares both value and type.
<code>console.log(5 > 3 > 2)</code>	false	JavaScript evaluates from left to right . First <code>5 > 3</code> becomes <code>true</code> . Then <code>true > 2</code> becomes <code>1 > 2</code> , which is false .
<code>console.log(true == 1)</code>	true	Using <code>==</code> , JavaScript performs type coercion . <code>true</code> is converted to <code>1</code> , so the comparison becomes <code>1 == 1</code> .

Object and String Comparisons

Expression	Output	Explanation
<code>console.log([] === [])</code>	false	Arrays are objects in JavaScript . Even if two arrays contain the same values, they are different objects in memory , so strict comparison returns false.
<code>console.log([1,2] == [1,2])</code>	false	Objects and arrays are compared by reference, not by value . Since both arrays are different instances, the result is false.
<code>console.log("10" < "9")</code>	true	Strings are compared lexicographically (Unicode order) . Since "1" comes before "9", "10" is considered smaller than "9".

Special JavaScript Values

Expression	Output	Explanation
<code>console.log(NaN === NaN)</code>	false	NaN means Not-a-Number . According to IEEE 754 rules, NaN is not equal to itself .
<code>console.log(undefined > 0)</code>	false	When compared with numbers, undefined converts to NaN . Any comparison involving NaN returns false .
<code>console.log("5" === 5)</code>	false	The strict equality operator (<code>===</code>) checks both value and data type . "5" is a string while 5 is a number.
<code>console.log(Infinity > 1000)</code>	true	Infinity represents an unbounded value , which is greater than any finite number.

Control Statements

Type	Purpose	Example
If / If-Else	Decision making	Checking positive or negative numbers
Switch	Multiple conditions	Selecting among many options
Ternary Operator	Short form of if-else	Quick condition checks
For Loop	Repeating a fixed number of times	Printing numbers
While Loop	Repeating while condition is true	Counting loops
Do-While Loop	Executes at least once	Menu-driven programs

Control Statement	Example	Output
if	<pre>let x = 5; if(x > 0) console.log("Positive");</pre>	Positive
if-else	<pre>let x = -3; if(x > 0) console.log("Positive"); else console.log("Negative");</pre>	Negative
switch	<pre>let day = 2; switch(day){case 1: console.log("Monday"); break; case 2: console.log("Tuesday");}</pre>	Tuesday
ternary operator	<pre>let age = 18; let result = age >= 18 ? \"Adult\" : \"Minor\"; console.log(result);</pre>	Adult
for loop	<pre>for(let i=1;i<=3;i++){ console.log(i);}</pre>	1 2 3
while loop	<pre>let i=1; while(i<=3){ console.log(i);\n i++;}</pre>	1 2 3
do-while loop	<pre>let i=1; do{ console.log(i); i++;while(i<=3);</pre>	1 2 3 12

Creating and Accessing Arrays in JavaScript

```
let a = ["HTML","CSS","JS"];  
  
console.log(a[0]);  
console.log(a[1]);
```

Expression	Output
a[0]	HTML
a[1]	CSS

Adding Elements to Array

Method	Purpose	Example	Result
push()	Add element at end	a.push("Node.js")	Adds at end
unshift()	Add element at start	a.unshift("Web Dev")	Adds at beginning

['Web Dev','HTML','CSS','JS','Node.js']

Removing Elements from Array

Method	Removes From	Example
pop()	Last element	a.pop()
shift()	First element	a.shift()
splice()	Specific position	a.splice(1,2)

Original: [HTML, CSS, JS]
After pop(): [HTML, CSS]
After shift(): [CSS]

Advanced Array Operations in JavaScript

Concept	Code Example	Output	Explanation
Array Length	<pre>let a = ["HTML","CSS","JS"]; console.log(a.length);</pre>	3	length returns the total number of elements in the array.
Increase Array Length	<pre>let a = ["HTML","CSS","JS"]; a.length = 5; console.log(a);</pre>	<pre>['HTML','CSS','JS', ,]</pre>	Increasing length creates additional empty slots in the array.
Decrease Array Length	<pre>a.length = 2; console.log(a);</pre>	<pre>['HTML','CSS']</pre>	Reducing length removes elements beyond the specified index.
Iteration (for loop)	<pre>let a = ["HTML","CSS","JS"]; for(let i=0;i<a.length;i++){ console.log(a[i]); } </pre>	HTMLCSSJS	The loop starts at index 0 and prints each element until the array length is reached.
Iteration (forEach)	<pre>a.forEach(function(x){ console.log(x);});</pre>	HTMLCSSJS	forEach() executes a function once for each element in the array .

Advanced Array Operations in JavaScript

Concept	Code Example	Output	Explanation
Array Concatenation	<pre>let a=["HTML","CSS"]; let b=["JS","Node"]; let result=a.concat(b); console.log(result);</pre>	<pre>['HTML','CSS','JS', Node']</pre>	concat() merges two arrays and returns a new combined array .
Array to String	<pre>Let a=["HTML","CSS","JS"]; console.log(a.toString());</pre>	HTML,CSS,JS	toString() converts array elements into a comma-separated string .
Check Array Type	<pre>let a=["HTML","CSS","JS"]; console.log(typeof a);</pre>	object	In JavaScript, arrays are technically objects .
Check Array (Correct Method)	<pre>Array.isArray(a);</pre>	true	Array.isArray() correctly checks if a variable is an array.
Array Mistake Example	<pre>const a1=[5]; const a2=new Array(5);</pre>	[5][empty × 5]	[5] creates an array with value 5 , while new Array(5) creates 5 empty slots .

Javascript output on Screen

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Test</h2>

<script>
let a = ["HTML","CSS","JS"];
a.length = 5;
document.write(a);
</script>

</body>
</html>
```

JavaScript Test

HTML,CSS,JS,,

JavaScript Code

```
<!DOCTYPE html>
<html>
<body>
<h1>My First JavaScript</h1>
<button type="button"
onclick="document.getElementById('demo').innerHTML = Date()"> Click me to display
Date and Time.</button>
<p id="demo"></p>
</body>
</html>
```

My First JavaScript

Click me to display Date and Time.

My First JavaScript

Click me to display Date and Time.

Sun Mar 15 2026 23:05:59 GMT+0530 (India Standard Time)